

Matrix Multiplication and Tiled Matrix Multiplication

Next Application: Matrix Multiplication

- Given two Width × Width matrices, M and N,
 - we can multiply M by N
 - to compute a third Width × Width matrix, P:
 - $P = MN$

In terms of the elements of P, matrix multiplication implies computing...

$$P_{ij} = \sum_{k=1}^{Width} M_{ik} N_{kj}$$

© Steven S. Lumetta, 2020
ECE408/CS483/University of Illinois at Urbana-Champaign

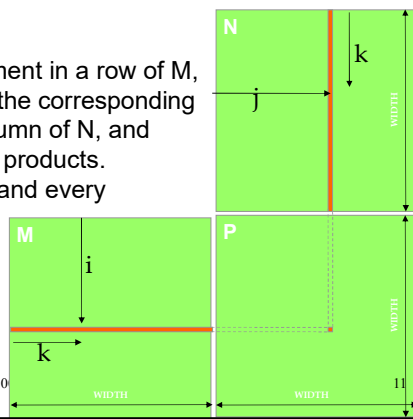
10

10

Matrix Multiplication

$$P_{ij} = \sum_{k=1}^{Width} M_{ik} N_{kj}$$

- Graphically, imagine
 - taking each element in a row of M,
 - multiplying it by the corresponding element in a column of N, and
 - summing up the products.
- Do that for every row and every column to produce P.



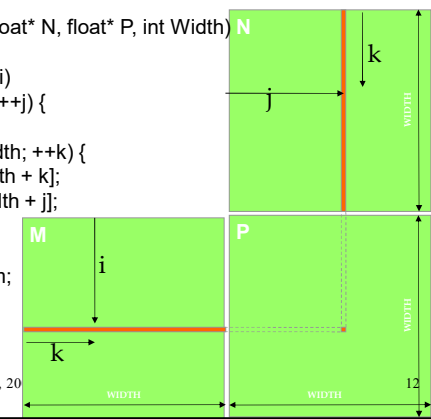
© David Kirk/NVIDIA and Wen-mei W. Hwu, 20
University of Illinois at Urbana-Champaign

11

Matrix Multiplication Example A Simple Host Version in C

// Matrix multiplication on the (CPU) host in single precision

```
void CPUMatMul(float* M, float* N, float* P, int Width)
{
    for (int i = 0; i < Width; ++i)
        for (int j = 0; j < Width; ++j) {
            float sum = 0;
            for (int k = 0; k < Width; ++k) {
                float a = M[i * Width + k];
                float b = N[k * Width + j];
                sum += a * b;
            }
            P[i * Width + j] = sum;
        }
}
```



© David Kirk/NVIDIA and Wen-mei W. Hwu, 20
University of Illinois at Urbana-Champaign

12

Parallelize Elements of P

- What can we parallelize?
 - start with the **two outer loops**
 - parallelize **computation of elements of P**
- What about the inner loop?
 - Technically, floating-point is NOT associative.
 - The **parallel sum** is called a **reduction**—we'll come back to it in a few weeks.
 - For now, **use a single thread for each P_{ij}** .

© Steven S. Lumetta, 2020
ECE408/CS483/University of Illinois at Urbana-Champaign

13

13

Tricky Questions about Floating-Point

A question for you:

What is $2^{-30} + (1 - 1)$?

Quite tricky, I know. But yes, it's **2^{-30}** .

Another question for you:

What is $(2^{-30} + 1) - 1$?

That's right. It's **0**.

At least it is with floating-point.

ECE 120: Introduction to Computing

© 2016 Steven S. Lumetta. All rights reserved. 14

14

IEEE Floating-Point is Not Associative

Why?

Take CS357: Numerical Methods

Our first sum was **$(2^{-30} + 1)$** .

To hold the integer 1, the bit pattern's exponent must be **2^0** .

But, the mantissa for single-precision floating point has only **23 bits**.

And thus represents powers down to **2^{-23}** .

The **2^{-30}** term is lost, giving **$(2^{-30} + 1) = 1$** .

So $2^{-30} + (1 - 1) \neq (2^{-30} + 1) - 1$.

ECE 120: Introduction to Computing

© 2016 Steven S. Lumetta. All rights reserved. 15

15

Compute Using 2D Blocks in a 2D Grid

- **P** is 2D, so organize threads in 2D as well:
 - Split the output **P** into square **tiles**
 - of size **$\text{TILE_WIDTH} \times \text{TILE_WIDTH}$**
 - (a preprocessor constant).
 - **Each thread block produces one tile** of TILE_WIDTH^2 elements.
 - Create **$\lceil \text{Width} / \text{TILE_WIDTH} \rceil^2$** thread **blocks** to cover the output matrix.

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/University of Illinois at Urbana-Champaign

16

16

Example: Width 8, TILE_WIDTH 2

P _{0,0}	P _{0,1}	P _{0,2}	P _{0,3}	P _{0,4}	P _{0,5}	P _{0,6}	P _{0,7}
P _{1,0}	P _{1,1}	P _{1,2}	P _{1,3}	P _{1,4}	P _{1,5}	P _{1,6}	P _{1,7}
P _{2,0}	P _{2,1}	P _{2,2}	P _{2,3}	P _{2,4}	P _{2,5}	P _{2,6}	P _{2,7}
P _{3,0}	P _{3,1}	P _{3,2}	P _{3,3}	P _{3,4}	P _{3,5}	P _{3,6}	P _{3,7}
P _{4,0}	P _{4,1}	P _{4,2}	P _{4,3}	P _{4,4}	P _{4,5}	P _{4,6}	P _{4,7}
P _{5,0}	P _{5,1}	P _{5,2}	P _{5,3}	P _{5,4}	P _{5,5}	P _{5,6}	P _{5,7}
P _{6,0}	P _{6,1}	P _{6,2}	P _{6,3}	P _{6,4}	P _{6,5}	P _{6,6}	P _{6,7}
P _{7,0}	P _{7,1}	P _{7,2}	P _{7,3}	P _{7,4}	P _{7,5}	P _{7,6}	P _{7,7}

Each block has
2*2 = 4 threads.

WIDTH/TILE_WIDTH = 4
Use 4*4 = 16 blocks.

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

17

17

Example: Same Matrix, Larger Tiles (Width 8, TILE_WIDTH 4)

P _{0,0}	P _{0,1}	P _{0,2}	P _{0,3}	P _{0,4}	P _{0,5}	P _{0,6}	P _{0,7}
P _{1,0}	P _{1,1}	P _{1,2}	P _{1,3}	P _{1,4}	P _{1,5}	P _{1,6}	P _{1,7}
P _{2,0}	P _{2,1}	P _{2,2}	P _{2,3}	P _{2,4}	P _{2,5}	P _{2,6}	P _{2,7}
P _{3,0}	P _{3,1}	P _{3,2}	P _{3,3}	P _{3,4}	P _{3,5}	P _{3,6}	P _{3,7}
P _{4,0}	P _{4,1}	P _{4,2}	P _{4,3}	P _{4,4}	P _{4,5}	P _{4,6}	P _{4,7}
P _{5,0}	P _{5,1}	P _{5,2}	P _{5,3}	P _{5,4}	P _{5,5}	P _{5,6}	P _{5,7}
P _{6,0}	P _{6,1}	P _{6,2}	P _{6,3}	P _{6,4}	P _{6,5}	P _{6,6}	P _{6,7}
P _{7,0}	P _{7,1}	P _{7,2}	P _{7,3}	P _{7,4}	P _{7,5}	P _{7,6}	P _{7,7}

Each block has
4*4 = 16 threads.

WIDTH/TILE_WIDTH = 2
Use 2*2 = 4 blocks.

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
University of Illinois at Urbana-Champaign

18

18

Kernel Invocation (Host-side Code)

```
// TILE_WIDTH is a #define constant
dim3 dimGrid(ceil((1.0*Width)/TILE_WIDTH),
             ceil((1.0*Width)/TILE_WIDTH), 1);
dim3 dimBlock(TILE_WIDTH, TILE_WIDTH, 1);

// Launch the device computation threads!
MatrixMulKernel<<<dimGrid, dimBlock>>>
    (Md, Nd, Pd, Width);
```

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

19

19

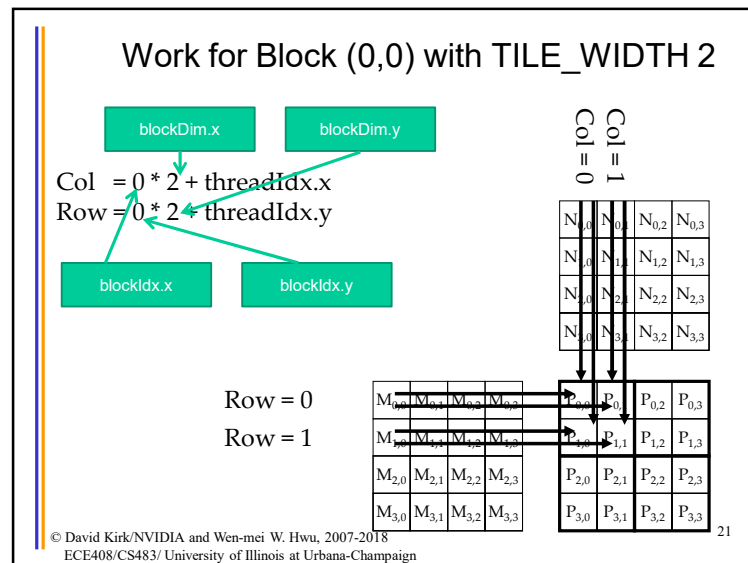
Kernel Function

```
// Matrix multiplication kernel – per thread code
__global__ void MatrixMulKernel(float* d_M, float* d_N, float* d_P, int Width)
{
    // Pvalue is used to store the element of the matrix
    // that is computed by the thread
    float Pvalue = 0;
```

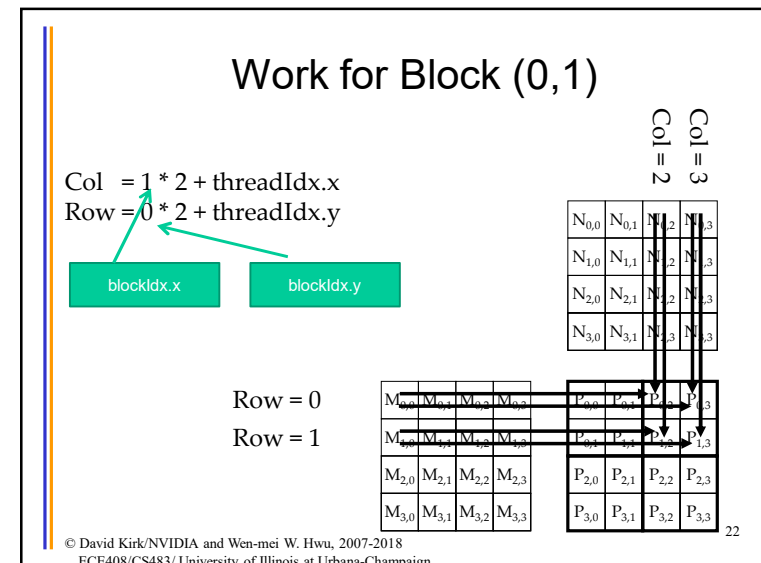
© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

20

20



21



22

A Simple Matrix Multiplication Kernel

```

global__ void MatrixMulKernel(float* d_M, float* d_N, float* d_P, int Width)
{
    // Calculate the row index of the d_P element and d_M
    int Row = blockIdx.y*blockDim.y+threadIdx.y;
    // Calculate the column idenx of d_P and d_N
    int Col = blockIdx.x*blockDim.x+threadIdx.x;

    if ((Row < Width) && (Col < Width)) {
        float Pvalue = 0;
        // each thread computes one element of the block sub-matrix
        for (int k = 0; k < Width; ++k)
            Pvalue += d_M[Row*Width+k] *
                    d_N[k*Width+Col];
        d_P[Row*Width+Col] = Pvalue;
    }
}

```

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

23

Memory Bandwidth is Overloaded!

- That's a **simple implementation**:
 - GPU kernel is the **CPU code**
 - with the **outer loops replaced**
 - with per-thread **index calculations!**
- Unfortunately, performance is quite bad.
- Why?
- With the given approach,
 - global **memory bandwidth**
 - can't** supply enough data
 - to **keep the SMs busy!**

© Steven S. Lumetta, 2020
ECE408/CS483/ University of Illinois at Urbana-Champaign

24

Where Do We Access Global Memory?

```
global__ void MatrixMulKernel(float* d_M, float* d_N, float* d_P, int Width)
{
    // Calculate the row index of the d_P element and d_M
    int Row = blockIdx.y*blockDim.y+threadIdx.y;
    // Calculate the column index of d_P and d_N
    int Col = blockIdx.x*blockDim.x+threadIdx.x;

    if ((Row < Width) && (Col < Width)) {
        float Pvalue = 0;
        // each thread computes one element of the block sub-matrix
        for (int k = 0; k < Width; ++k)
            Pvalue += d_M[Row*Width+k] * d_N[k*Width+Col];
        d_P[Row*Width+Col] = Pvalue;
    }
}
```

accesses to global memory

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

25

25

Each Thread Requires 4B of Data per FLOP

- Each threads access global memory
 - for elements of **M** and **N**:
–**4B each**, or **8B per pair**.
–(And once TOTAL to **P** per thread—ignore it.)
- With each pair of elements,
 - a thread does a single multiply-add,
–**2 FLOP**—floating-point operations.
- So for every FLOP,
 - a thread needs 4B** from memory:
–**4B / FLOP**.

© Steven S. Lumetta, 2020
ECE408/CS483/ University of Illinois at Urbana-Champaign

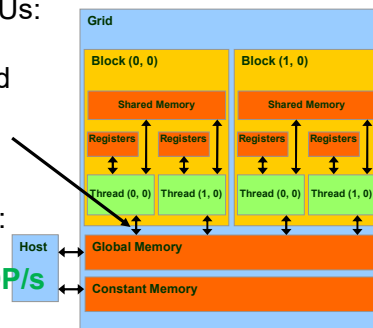
26

26

150 GB/s Bandwidth Implies 37.5 GFLOPs

- One generation of GPUs:
 - 1,000 GFLOP/s** of compute power, and
 - 150 GB/s** of memory bandwidth.
- Dividing bandwidth by memory requirements:

$$\frac{150 \text{ GB/s}}{4 \text{ B/FLOP}} = 37.5 \text{ GFLOP/s}$$
 which **limits computation!**



© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

27

27

What to Do? Reuse Memory Accesses!

But **37.5 GFLOPs is a limit**.

In an **actual execution**,

- memory is not busy all the time, and
- the code **runs at** about **25 GFLOPs**.

To get closer to 1,000 GFLOPs

- we **need to** drastically **cut down**
- **accesses to global memory**.

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

28

28

Review: Extremely Poor Performance

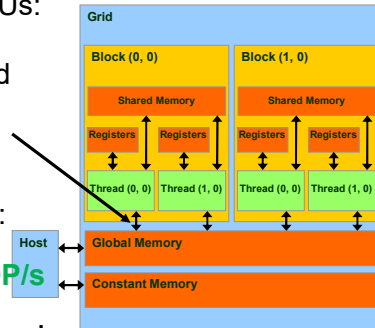
- One generation of GPUs:

- **1,000 GFLOP/s** of compute power, and
- **150 GB/s** of memory bandwidth.

- Dividing bandwidth by memory requirements:

$$\frac{150 \text{ GB/s}}{4 \text{ B/FLOP}} = 37.5 \text{ GFLOP/s}$$

which **limits computation!**



© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

5

The Solution? Reuse Memory Accesses!

But **37.5 GFLOPs** is a limit.

In an **actual execution**,

- memory is not busy all the time, and
- the code **runs at** about **25 GFLOPs**.

To get closer to 1,000 GFLOPs

- we **need to** drastically **cut down**
- accesses to global memory.**

But ... how?

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
University of Illinois at Urbana-Champaign

6

Tiled Matrix-Matrix Multiplication using Shared Memory

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
University of Illinois at Urbana-Champaign

7

A Common Programming Strategy

- The dilemma:
 - Matrices **M** and **N** are large.
 - They **fit** easily **in global memory**, but that's **slow**.
 - Shared memory** is **fast**, but **M and N don't fit**.
- The solution:
 - Break M and N into tiles**
 - (called blocks in the much older CPU literature).
 - Read a tile** into shared memory.
 - Use the tile** from shared memory.
 - Repeat** until done.

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
University of Illinois at Urbana-Champaign

8

8

A Common Programming Strategy

- In a GPU, **only threads in a block** can use **shared** memory.
- Thus, each **block** operates on **separate tiles**:
 - **Read tile(s)** into shared memory **using multiple threads** to exploit memory-level parallelism.
 - **Compute** based on shared memory tiles.
 - **Repeat.**
 - **Write results** back **to global memory**.

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
University of Illinois at Urbana-Champaign

9

9

Declaring Shared Memory Arrays

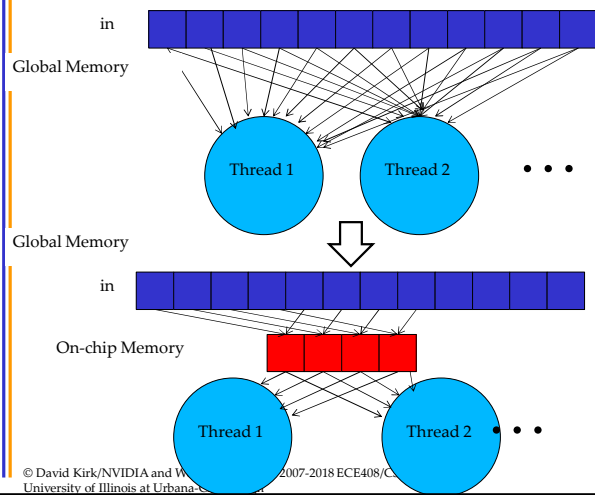
```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width)
{
    __shared__ float subTileM[TILE_WIDTH][TILE_WIDTH];
    __shared__ float subTileN[TILE_WIDTH][TILE_WIDTH];
}
```

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/University of Illinois at Urbana-Champaign

10

10

Shared Memory Tiling Basic Idea



© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
University of Illinois at Urbana-Champaign

11

11

Outline of Technique

- Identify a tile of global data that are accessed by multiple threads
- Load the tile from global memory into on-chip memory
- Have the multiple threads to access their data from the on-chip memory
- Move on to the next block/tile

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
University of Illinois at Urbana-Champaign

12

12

Idea: Place global memory data into Shared Memory for **reuse**

- Each input element is used to calculate **WIDTH** elements of **P**.
- Load each element into Shared Memory and have several threads use the local version to reduce memory bandwidth.

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
University of Illinois at Urbana-Champaign

13

Tiled Multiply

- Break up the execution of the kernel into phases so that the data accesses in each phase are focused on one subset (tile) of **M** and **N**

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/University of Illinois at Urbana-Champaign

14

Loading a Tile

- All threads in a block participate
 - Each thread loads
 - one **M** element and
 - one **N** element
 - in basic tiling code.
- Assign the loaded element to each thread such that the accesses within each warp is coalesced (more later).

15

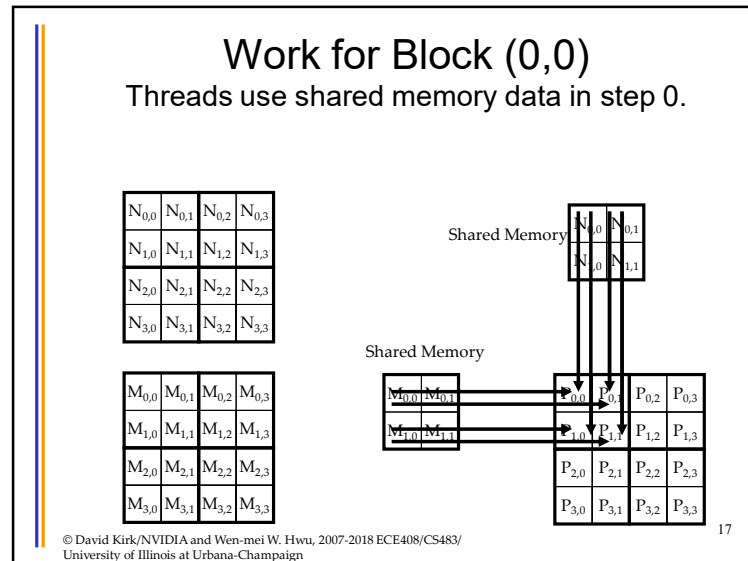
© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
University of Illinois at Urbana-Champaign

15

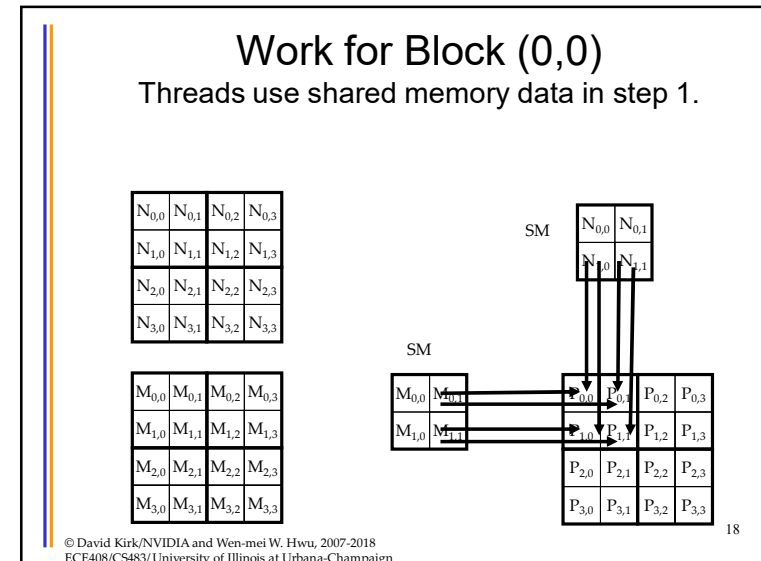
Work for Block (0,0)

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/University of Illinois at Urbana-Champaign

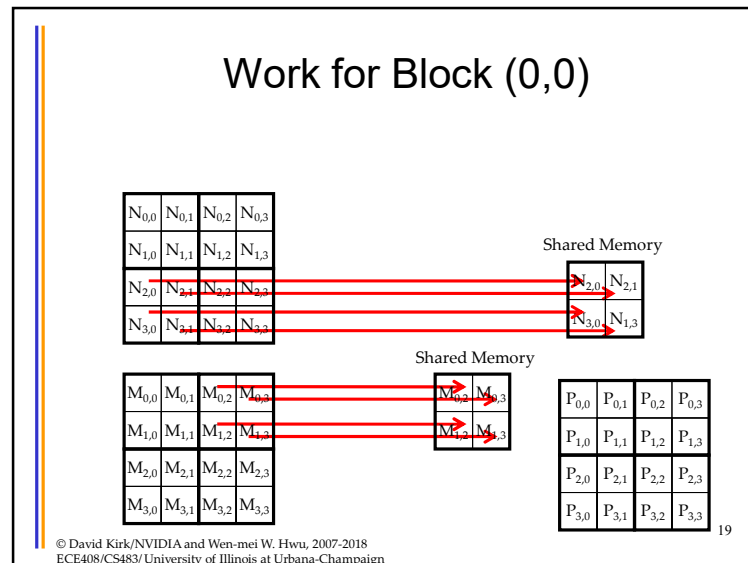
16



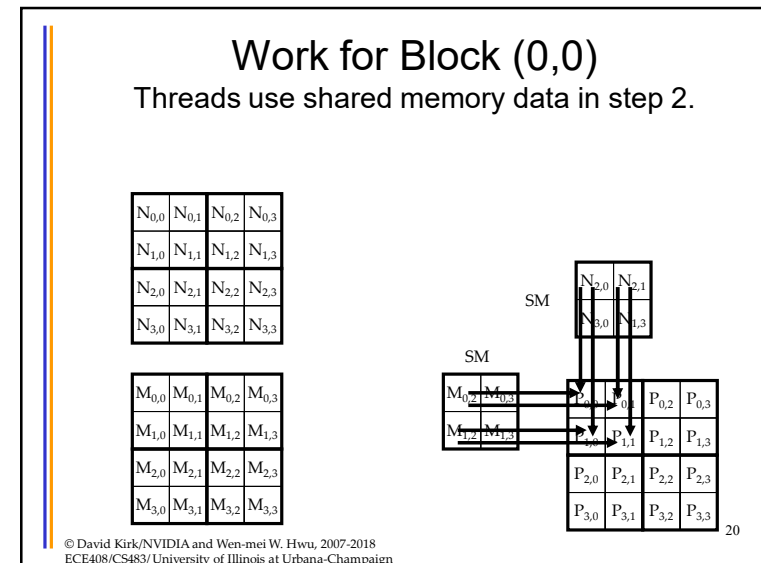
17



18



19

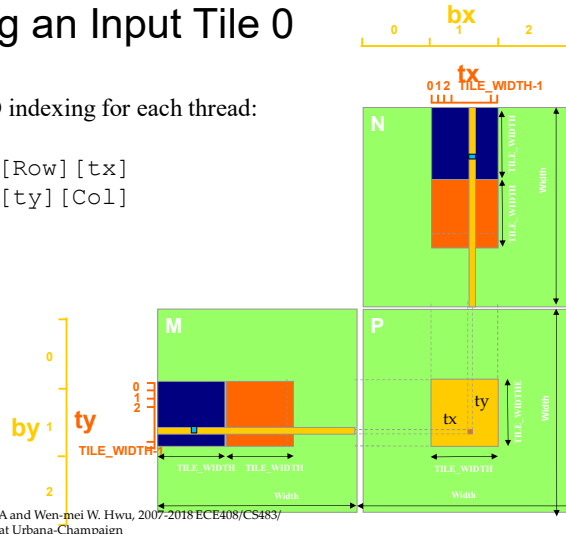


20

Loading an Input Tile 0

Tile 0 2D indexing for each thread:

$M[\text{Row}][\text{tx}]$
 $N[\text{ty}][\text{Col}]$



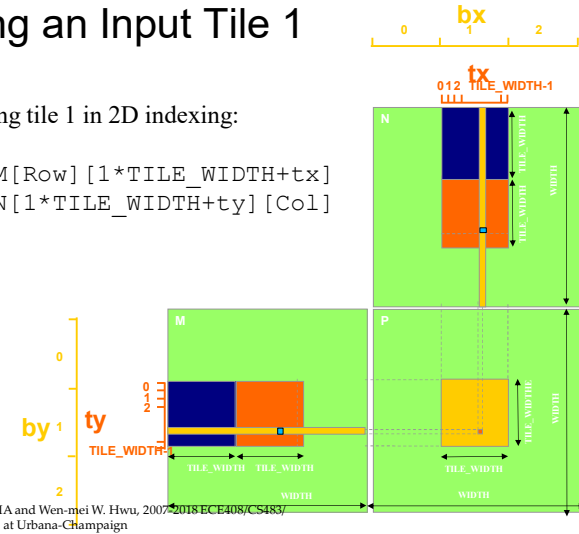
© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
 University of Illinois at Urbana-Champaign

21

Loading an Input Tile 1

Accessing tile 1 in 2D indexing:

$M[\text{Row}][1 * \text{TILE_WIDTH} + \text{tx}]$
 $N[1 * \text{TILE_WIDTH} + \text{ty}][\text{Col}]$



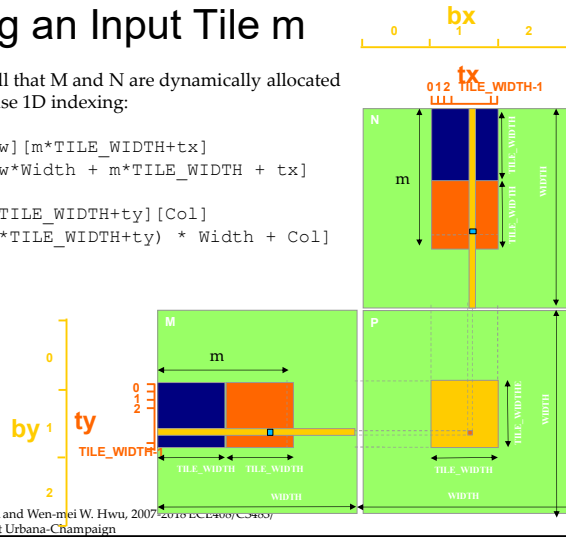
© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
 University of Illinois at Urbana-Champaign

22

Loading an Input Tile m

However, recall that M and N are dynamically allocated
 and can only use 1D indexing:

$M[\text{Row}][m * \text{TILE_WIDTH} + \text{tx}]$
 $M[\text{Row} * \text{Width} + m * \text{TILE_WIDTH} + \text{tx}]$
 $N[m * \text{TILE_WIDTH} + \text{ty}][\text{Col}]$
 $N[(m * \text{TILE_WIDTH} + \text{ty}) * \text{Width} + \text{Col}]$



© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
 University of Illinois at Urbana-Champaign

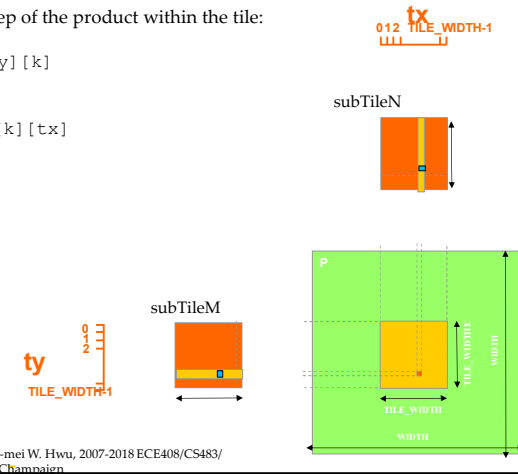
23

Accessing a Tile

To perform the k^{th} step of the product within the tile:

$\text{subTileM}[\text{ty}][k]$

$\text{subTileN}[k][\text{tx}]$



© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
 University of Illinois at Urbana-Champaign

24

We're Not There Yet!

- But ...
- **How can a thread know ...**
 - **That another thread has finished** its part of the tile?
 - Or that another thread has finished using the previous tile?

We need to synchronize!

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
University of Illinois at Urbana-Champaign

25

25

Leveraging Parallel Strategies

- **Bulk synchronous execution:**
threads execute roughly in unison
 1. Do some work
 2. Wait for others to catch up
 3. Repeat
- **Much easier programming model**
 - Threads only parallel within a section
 - Debug lots of little programs
 - Instead of one large one.
- **Dominates high-performance applications**

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
University of Illinois at Urbana-Champaign

26

26

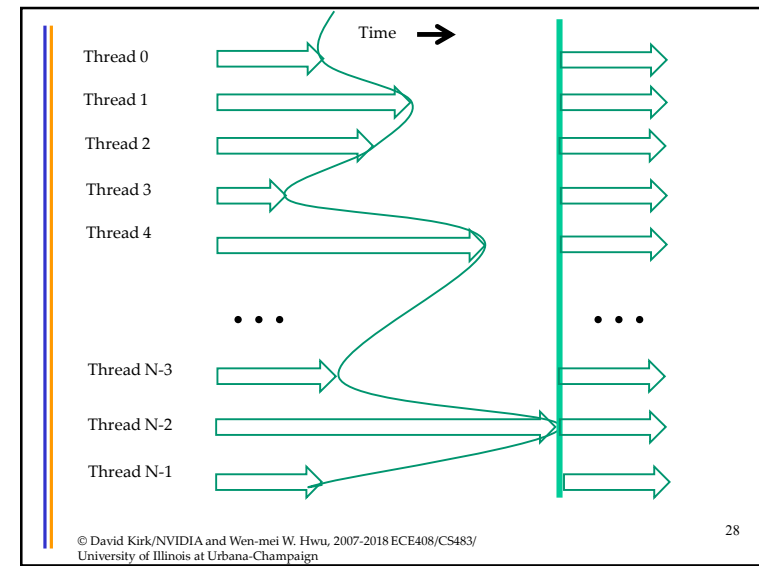
Bulk Synchronous Steps Based on Barriers

- **How does it work?**
Use a barrier to wait for thread to 'catch up.'
- A barrier is a synchronization point:
 - **each thread calls a function** to enter barrier;
 - **threads block** (sleep) in barrier function **until all threads have called**;
 - **after last thread calls** function, **all threads continue** past the barrier.

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
University of Illinois at Urbana-Champaign

27

27



© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
University of Illinois at Urbana-Champaign

28

28

Use __syncthreads for CUDA Blocks

- **How does it work in CUDA?**
Only **within thread blocks**!
- The function: **void __syncthreads(void) ;**
- N.B.
 - **All threads** in block **must enter** (no subsets).
 - All threads must enter the **SAME static call** (not the same as all threads calling function!).

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
University of Illinois at Urbana-Champaign

29

29

Barrier Trauma: What's Actually Done?

- **What exactly is guaranteed** to have finished?
 - Are **shared memory** operations before a barrier (e.g., stores) guaranteed to have completed?
 - What about **global memory** ops?
 - What about **atomic ops** with no return values?
 - What about I/O operations?
- CUDA manual: all global and shared memory ops (which presumably includes atomic variants) have completed.
- **Avoid assumptions about I/O** (such as printf).

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
University of Illinois at Urbana-Champaign

30

30

Tiled Matrix Multiplication Kernel

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width)
{
    1. __shared__ float subTileM[TILE_WIDTH][TILE_WIDTH];
    2. __shared__ float subTileN[TILE_WIDTH][TILE_WIDTH];

    3. int bx = blockIdx.x; int by = blockIdx.y;
    4. int tx = threadIdx.x; int ty = threadIdx.y;

    // Identify the row and column of the P element to work on
    5. int Row = by * TILE_WIDTH + ty; // note: blockDim.x == TILE_WIDTH
    6. int Col = bx * TILE_WIDTH + tx; //      blockDim.y == TILE_WIDTH
    7. float Pvalue = 0;

    // Loop over the M and N tiles required to compute the P element
    // The code assumes that the Width is a multiple of TILE_WIDTH!
    8. for (int m = 0; m < Width/TILE_WIDTH; ++m) {
        // Collaborative loading of M and N tiles into shared memory
        9. subTileM[ty][tx] = M[Row*Width + m*TILE_WIDTH+tx];
        10. subTileN[ty][tx] = N[(m*TILE_WIDTH+ty)*Width+Col];
        11. __syncthreads();
        12. for (int k = 0; k < TILE_WIDTH; ++k)
        13.     Pvalue += subTileM[ty][k] * subTileN[k][tx];
        14. __syncthreads();
    15. }
    16. P[Row*Width+Col] = Pvalue;
}
```

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
University of Illinois at Urbana-Champaign

31

31

Compare with Basic MM Kernel

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width)
{
    // Calculate the row index of the P element and M
    int Row = blockIdx.y * blockDim.y + threadIdx.y;
    // Calculate the column index of P and N
    int Col = blockIdx.x * blockDim.x + threadIdx.x;

    if ((Row < Width) && (Col < Width)) {
        float Pvalue = 0;

        // each thread computes one element of the block sub-matrix
        for (int k = 0; k < Width; ++k)
            Pvalue += M[Row*Width+k] * N[k*Width+Col];

        P[Row*Width+Col] = Pvalue;
    }
}
```

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
University of Illinois at Urbana-Champaign

32

32

Use of Large Tiles Shifts Bottleneck

- Recall our example GPU: **1,000 GFLOP/s, 150 GB/s**
- 16x16 tiles** use each operand for 16 operations
 - **reduce global** memory **accesses by** a factor of **16**
 - **150GB/s** bandwidth supports $(150/4)*16 = \mathbf{600\ GFLOPS!}$
- 32x32 tiles** use each operand for 32 operations
 - **reduce global** memory **accesses by** a factor of **32**
 - **150 GB/s** bandwidth supports $(150/4)*32 = \mathbf{1,200\ GFLOPS!}$
 - **Memory bandwidth is no longer the bottleneck!**

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
University of Illinois at Urbana-Champaign

33

33

Also Need Parallel Accesses to Memory

- Shared memory size
 - implementation dependent
 - **64kB** per SM in Maxwell (48kB max per block)
- Given **TILE_WIDTH of 16** (256 threads / block),
 - each thread block uses $2*256*4B = 2kB$ of shared memory,
 - which limits active blocks to 32;
 - max. of 2048 threads per SM,
 - which limits blocks to 8.
 - Thus up to $8*512 = \mathbf{4,096\ pending\ loads}$ (2 per thread, 256 threads per block)

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
University of Illinois at Urbana-Champaign

34

34

Another Good Choice: 32x32 Tiles

- Given **TILE_WIDTH of 32** (1,024 threads / block),
 - each thread block uses $2*1024*4B = 8kB$ of shared memory,
 - which limits active blocks to 8;
 - max. of 2,048 threads per SM,
 - which limits blocks to 2.
 - Thus up to $2*2,048 = \mathbf{4,096\ pending\ loads}$ (2 per thread, 1,024 threads per block)

(same memory parallelism exposed)

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
University of Illinois at Urbana-Champaign

35

35

Current GPU? Use Device Query

- Number of devices in the system


```
int dev_count;
cudaGetDeviceCount( &dev_count);
```
- Capability of devices


```
cudaDeviceProp dev_prop;
for (i = 0; i < dev_count; i++) {
    cudaGetDeviceProperties( &dev_prop, i);

    // decide if device has sufficient resources and capabilities
}
```
- `cudaDeviceProp` is a built-in C structure type
 - `dev_prop.dev_prop.maxThreadsPerBlock`
 - `Dev_prop.sharedMemoryPerBlock`
 - ...

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018 ECE408/CS483/
University of Illinois at Urbana-Champaign

36

36